DETERMINING K CLUSTERS IN K-MEANS CLUSTERING WITH THE CRAB ALGORITHM

A Thesis

presented to

the Faculty of California Polytechnic State University,

San Luis Obispo

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Statistics

by

Jasmine Kristine Soriano Cabrera

June 2026

COMMITTEE MEMBERSHIP

TITLE:	Determining k Clusters in k-Means Clustering with the CRAB Algorithm
AUTHOR:	Jasmine Kristine Soriano Cabrera
DATE SUBMITTED:	June 2026

COMMITTEE CHAIR:	Kelly Bodwin, Ph.D. Professor of Statistics
COMMITTEE MEMBER:	Allison Theobold, Ph.D. Professor of Statistics
COMMITTEE MEMBER:	Julia Schedler, Ph.D. Professor of Statistics

ABSTRACT

Determining k Clusters in k-Means Clustering with the CRAB Algorithm

Jasmine Kristine Soriano Cabrera

Unsupervised clustering algorithms such as k-Means and Hierarchical Clustering are commonly used in research and academia. While many different scoring methods have been developed to determine the most optimal number of clusters (i.e. the Elbow Method, Silhouette Score, etc.), their vague results and initial assumptions about the data, present the need for a new approach in finding the optimal k number of clusters. To address this, a new cluster scoring method, the CRAB algorithm, was created. This approach works specifically with k-Means and utilizes random subsampling of the data along with the classification of its pairwise points. The constructed definition of "Buddies" and "Rivals" provides insight into which observation points tend to stick together and which tend to be further apart. For research and educational purposes, an R package is being built, implementing different functions to help understand the many capabilities of the cRab algorithm.

Keywords: Clustering, Data Science, k-Means.

ACKNOWLEDGMENTS

This paper is not only a reflection of the past four years of my education, but also all of the love and support placed in me by my family. I would first like to dedicate this to my Mom, my younger sister, Kirsten, my younger brother AJ, and most importantly, my Dad, who I miss every day.

I would also like to dedicate this to my second family found in Statistics Department, where we bonded over late nights in the Stat Lab to the Grad Lab.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

Chapter 1

INTRODUCTION

1.1 Motivation

Unsupervised learning is a type of machine learning where data is unlabeled and the true target variable is unknown. Practitioners often use it to identify groups of observations, reduce dimensionality, or detect anomalies within data. While these methods can reveal meaningful structure in data, evaluating their results is inherently difficult because no ground truth exists, unlike in supervised learning.

Clustering is the most widely recognized form of unsupervised learning, with K-means being one of its most commonly used algorithms. K-means starts by initializing k centroids in the data space and assigns each point to the nearest centroid based on Euclidean distance. It then updates the centroids as the mean of their assigned points and repeats this process until the cluster assignments stabilize. Despite its simplicity, K-means has notable limitations. One flaw is the final clustering depends heavily on the initial placements of the centroids (Jaeger), which can lead to inconsistent or suboptimal results. Additionally, the use of Euclidean distance assumes that the clusters themselves are roughly spherical, an assumption that often fails for real-world data.

A major challenge in clustering as a whole is determining the appropriate number of clusters. Without prior knowledge, this choice can significantly affect the outcome, and different methods often suggest different answers. Two of the most commonly used techniques are the Elbow Method and the Silhouette Score.

1.2 Current Methods

The Elbow Method is a visual approach that evaluates clustering by calculating the within-cluster sum of squares (WCSS). This measure sums the squared Euclidean distances between each point and its assigned cluster centroid for different values of k. As the number of clusters increases, the WCSS naturally decreases because smaller clusters reduce within-cluster variation. Analysts typically identify the optimal number of clusters at the point where the decrease in WCSS slows dramatically and begins to level off, forming what is known as the "elbow". They then choose the value of k just before this plateau as the correct number of clusters. Although this method works well when the elbow is clearly visible, it does not always provide a clear answer (Kodinariya).

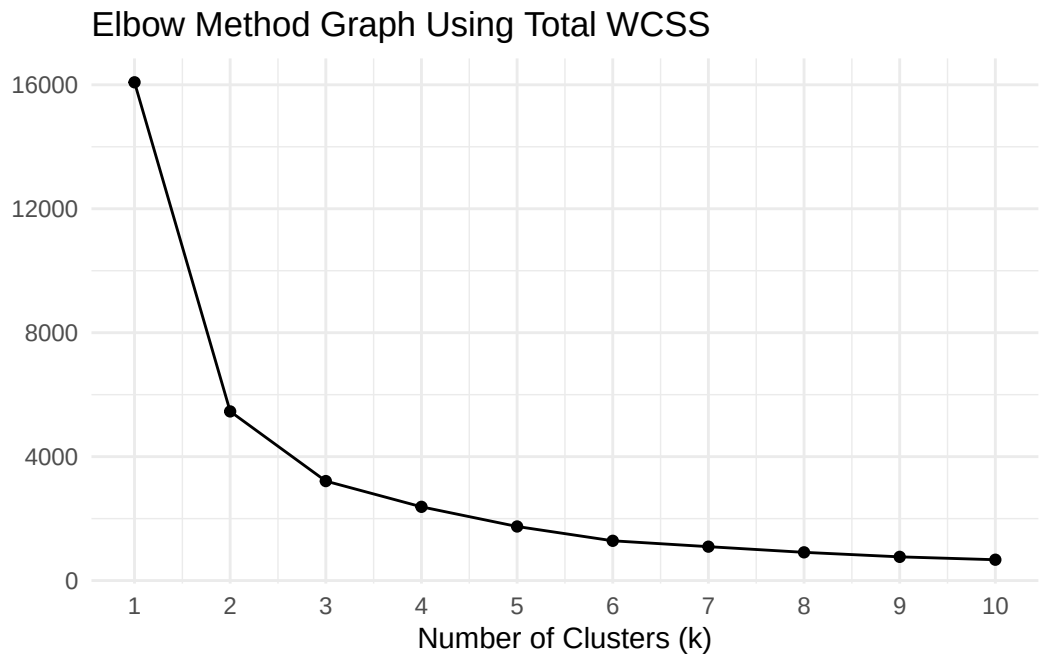

Figure 1.1: Elbow Method Graph using WCSS

Figure 1.1 illustrates a case where WCSS decreases gradually for all values of k, making it difficult to identify a distinct elbow. Without a clear plateau to guide the decision, users must select the optimal number of clusters subjectively, which can produce very different results depending on the chosen value of k.

In contrast to the Elbow Method, the Silhouette Score provides a more quantitative approach by removing the subjectivity of visual interpretation. Instead of relying on a plot, it judges clusters using a mathematical measure and selects the number of clusters based on the global maximum which corresponds to well separated and cohesive clusters. However, this method introduces its own limitation due to the assumptions it makes about cluster structure.

The Silhouette Score is defined as:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ is the average distance between point i and all other points in the same cluster, and $b(i)$ is the average distance between point i and points in the nearest neighboring cluster.

Intuitively, this measure evaluates how well a point fits within its assigned cluster compared to others. Because of its formulation, the Silhouette Score favors compact and well separated clusters. As a result, it performs well when clusters are roughly uniform in shape, but struggles with oddly shaped or elongated structures that real data tends to resemble. Additionally, maximizing the global score can bias the method toward a solution with fewer clusters, prioritizing separation over accurately capturing the underlying structure. This limitation is illustrated in Figures 1.2 and 1.3.

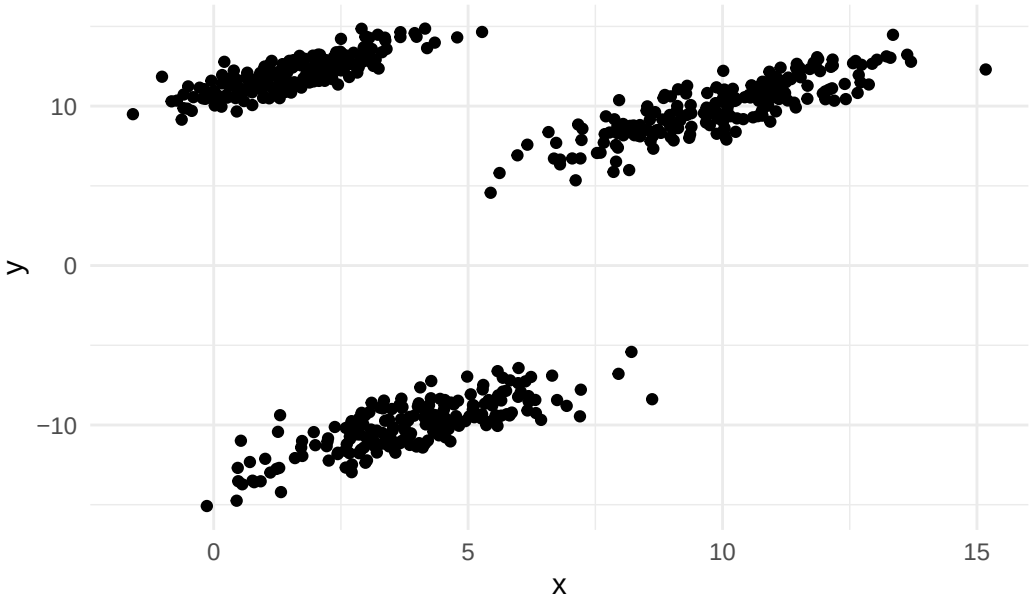

Figure 1.2: Silhouette Score Data

Figure 1.3: Silhouette Score Graph

Although the dataset visually contains three distinct clusters, the Silhouette Score identifies $k = 2$ as the most optimal. This occurs because the two upper clusters are relatively close to

each other compared to the lower cluster, leading the algorithm to favor greater separation over correctly identifying all three groups.

1.3 Previous Research

These drawbacks have motivated alternative approaches that go beyond evaluating clusters based solely on distance or compactness. One method introduced by Asa Ben-Hur and colleagues, evaluates clustering through stability. Their approach stems from the idea that a clustering into k groups captures the inherent structure if similar cluster assignments are obtained across various subsamples of the data. To assess this, the method repeatedly clusters subsets of the data using a hierarchical clustering method and compares the consistency of assignments between observations found in both subsets. This method measures similarity between clusters using cosine similarity, producing a distribution of similarity scores for each value of k. When a true underlying structure exists, these scores remain consistently high and concentrated near 1, indicating that the clustering is stable across perturbations of the data.

1.4 CRAB Algorithm

Building on this idea, the CRAB algorithm evaluates clustering quality through stability rather than relying on assumptions about cluster shape or separation. It repeatedly subsamples and reclusters the data to measure how consistently pairwise observations are assigned to the same cluster across different samples. This approach shifts the focus from how the clusters look visually to their reproducibility, providing stronger evidence that the identified groupings are meaningful.

Unlike the Elbow Method and Silhouette Score, the CRAB algorithm makes no assumptions about the data or group structure, allowing it to better capture more complex and irregular patterns. Additionally, its use of a global minimum provides a clear and objective selection

criterion, avoiding the ambiguity of visual interpretation and the bias toward compact clusters. Although this approach may require additional computation due to repeated sampling, it offers a more reliable way to determine the number of clusters.

The following chapters expand on this method. Chapter 2 provides a more detailed mathematical explanation, while Chapter 3 introduces an R package with functions that demonstrate the capabilities and applications of the cRab algorithm on both real and simulated data.

Chapter 2

PROPERTIES AND DETAILS

2.1 Overview

The CRAB algorithm, which stands for Clustering Rivals and Buddies, is designed to evaluate clustering performance by examining how consistently points are grouped together across multiple subsamples of the data. Unlike traditional methods that rely on assumptions about cluster shape, size, or separation, CRAB emphasizes reproducibility, providing a more robust measure. At its core, CRAB focuses on pairwise relationships between observations. By tracking how often two points end up in the same cluster across repeated resampling, the algorithm quantifies the stability of cluster assignments. This approach allows it to handle irregular clusters that would challenge methods such as the Elbow Method and Silhouette Score. The output is a single metric, the CRAB score, which provides a clear answer for selecting the most appropriate number of clusters while minimizing subjectivity.

cRab Algorithm:

Inputs:

data: Dataset to analyze
min_k: Starting number of clusters k to evaluate
max_k: Ending number of clusters k to evaluate
subsample_prop: Fraction of the dataset to use for each resample
num_subsamples: Number of resamples to generate
cluster_method: Clustering algorithm to apply (i.e. K-means)

Output: A data frame where each row corresponds to a tested k cluster. The data frame includes the CRAB score, runtime, and a "Best Cluster" column. Only the row corresponding to the lowest CRAB score is marked as "BEST" while all other rows remain blank.

For all desired values of k:

1. **Subsample the data:** Randomly select f proportion of the observations.
2. **Cluster the subsample:** Apply the chosen clustering method to assign cluster labels to each point in subsample.
3. **Record pairwise assignments:** Assign Buddies and Rivals for each pair.
   - **Buddies (+1):** In the same cluster
   - **Rivals (-1):** In different clusters
   - **Ignore (0):** One or both are not in subsample
4. **Repeat resampling and clustering:** Perform steps 1-3 for a total of r resamples, generating a Buddies Matrix for each iteration.
5. **Aggregate pairwise stability:** Combine the Buddies Matrices into a Mean Matrix to measure how consistently each pair of points clusters together across resamples.
6. **Compute CRAB score:** Calculate the average squared distance from 1 across all valid Mean Matrix entries to produce a single cluster stability metric.
7. **Select optimal k:** Choose k with the lowest CRAB score as the most stable clustering solution.

Figure 2.1: CRAB Algorithm

2.2 Preparing the Data

Before applying the CRAB algorithm, ensure that the dataset is preprocessed appropriately for the chosen clustering method. For example, if using K-means, this may include handling missing values and standardizing features so that the variables on different scales do not

disproportionately influence the clustering algorithm.

The CRAB algorithm requires three main inputs:

- **Subsample Proportion (f):** Fraction of the dataset to use for each resample
- **Number of resamples (r):** Number of resamples to generate (i.e. number of times the data will be subsampled)
- **Range of k values:** The different cluster counts to evaluate

For each resample, CRAB draws a random subset of the data containing f proportion of the observations and applies the chosen clustering method (i.e. K-means) to assign cluster labels to each point. By repeating this process across multiple resamples, the algorithm detects which cluster assignments are consistently reproduced, instead of placing confidence on any single clustering of the full dataset.

2.3 Buddies Matrix

After clustering each resample, CRAB constructs a Buddies Matrix, B, to track pairwise relationships between observations. This $n \times n$ matrix encodes whether points i and j were assigned to the same cluster:

$$B_{i,j} = \begin{cases} 1, & \text{if they are in the same cluster} \\ -1, & \text{if they are in different clusters} \\ 0, & \text{if one or both points are not included in the subsample} \end{cases}$$

To reduce computation and take advantage of symmetry, CRAB sets the diagonal and lower triangle of each Buddies Matrix to NA. Each resample produces one Buddies Matrix, resulting in r matrices in total. Collectively, these matrices capture how pairwise relationships vary across resamples, forming the foundation for measuring cluster stability. Ideally, each pair

of observations (i, j) should consistently have either 1s (always assigned to the same cluster) or -1s (always assigned to different clusters) across resamples. Pairs that switch between 1 and -1 indicate instability in the clustering assignments.

For example, Figure 2.1 shows a random sample from a mock dataset clustered using K-means. The five labeled points illustrate all possible scenarios: Observations 1 and 3 belong in different clusters, while Observations 4 and 5 are in the same cluster. Observation 2, although part of the original dataset, was not included in this particular sample.

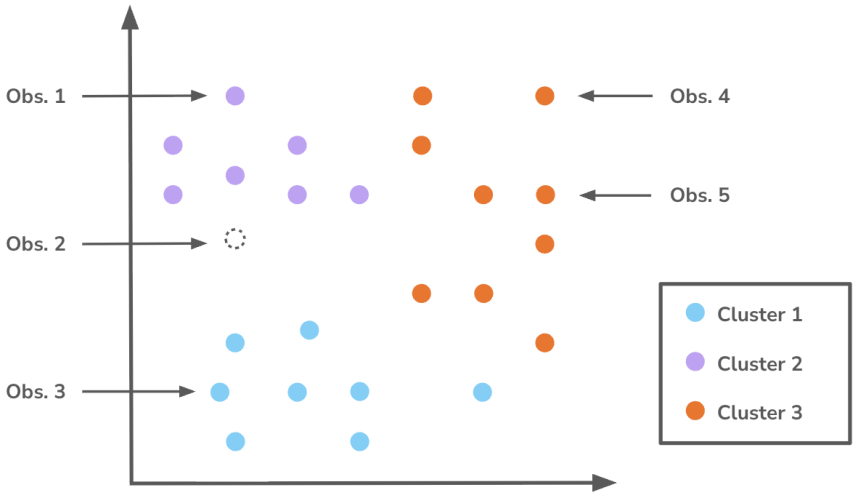

Figure 2.2: Buddies Matrix Data

Figure 2.2 shows how these pairwise relationships are represented in a Buddies Matrix.

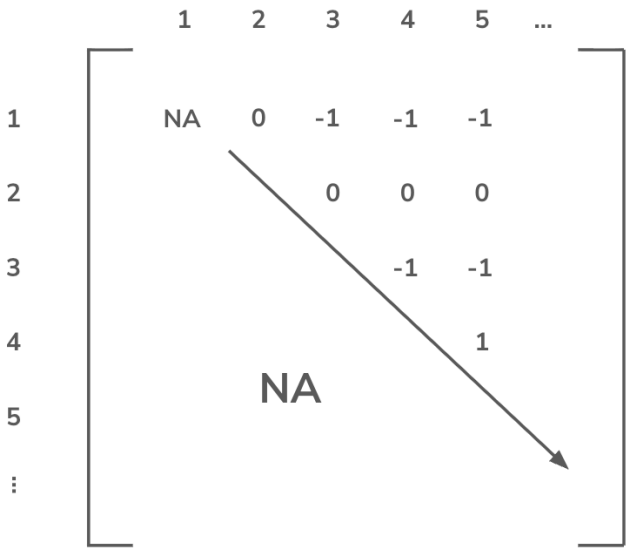

Figure 2.3: Buddies Matrix Example

2.4 Mean Matrix

The algorithm aggregates the Buddies Matricies to form the Mean Matrix, which summarizes the overall stability of pairwise cluster assignments. For each entry (i, j), it first counts the number of resamples where both points were included. It then sums the corresponding entries from thpse Buddies Matrices and divides by the number of comparisons to obtain the mean. Each entry ranges from -1 and 1, reflecting how consistently the pair of points clusters together across resamples. Values closer to -1 suggest that the points are often placed in different clusters, while values closer to 1 indicate that the points are frequently assigned to the same cluster. Entries near 0 indicate inconsistent or ambiguous pairwise assignments. The Mean Matrix acts as a stability map, revealing which relationships remain robust across different subsamples of the data. Figure 2.3 visualy demonstrates how the algorithm constructs the Mean Matrix from the Buddies Matrices.

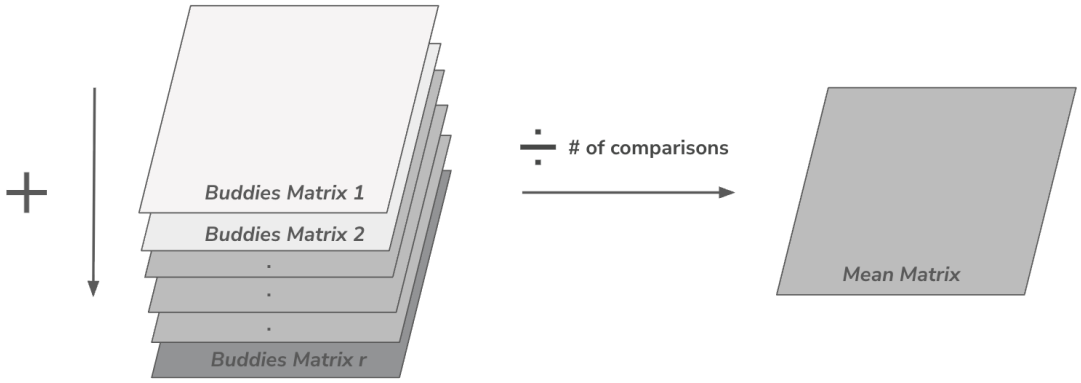

Figure 2.4: Mean Matrix Example

2.5 Similarity Score

To summarize the information in the Mean Matrix, the CRAB score quanitifes cluster stability. Let M represent the Mean Matrix with entries $M_{i,j}$ for each pair of observations. The CRAB score is defined as:

$$\text{CRAB Score} = \frac{1}{|V|} \sum_{(i,j) \in V} (1 - |M_{i,j}|)^2$$

Where:

- V is the set of all valid entries in the vectorized Mean Matrix (i.e. all pairs that are not NA)
- $|V|$ is the number of valid entries
- $M_{i,j}$ is the Mean Matrix entry for the i, j pair

Scores close to 0 indicate that points consistently group together across all subsamples, suggesting highly reproducible clusters. In contrast, higher scores denote greater instability, signaling that cluster assignments vary significantly depending on the subsample. The CRAB score condenses these various pairwise relationships into a single metric that can be

compared across different values of k.

2.6 CRAB Performance

By computing the CRAB scores across different values of k, the algorithm identifies the most stable clustering as the one with the lowest score. This provides an objective criterion for selecting the number of clusters. In many cases, when visualized across increasing k, the optimal value appears as a "pinching point" in the plot, where the score decreases before rising again for larger k values.

To illustrate the performance of CRAB, we use the well-known Palmer Penguins dataset (cite) as an effective benchmark. This dataset is particularly useful because it contains a true class label, species, allowing for direct comparison between identified clusters and the actual groupings. There are three unique species along with several physical measurements, such as bill length, bill depth, and flipper length. For this analysis, we select flipper length and bill length, as these features provide strong separation between species with minimal overlap among observations.

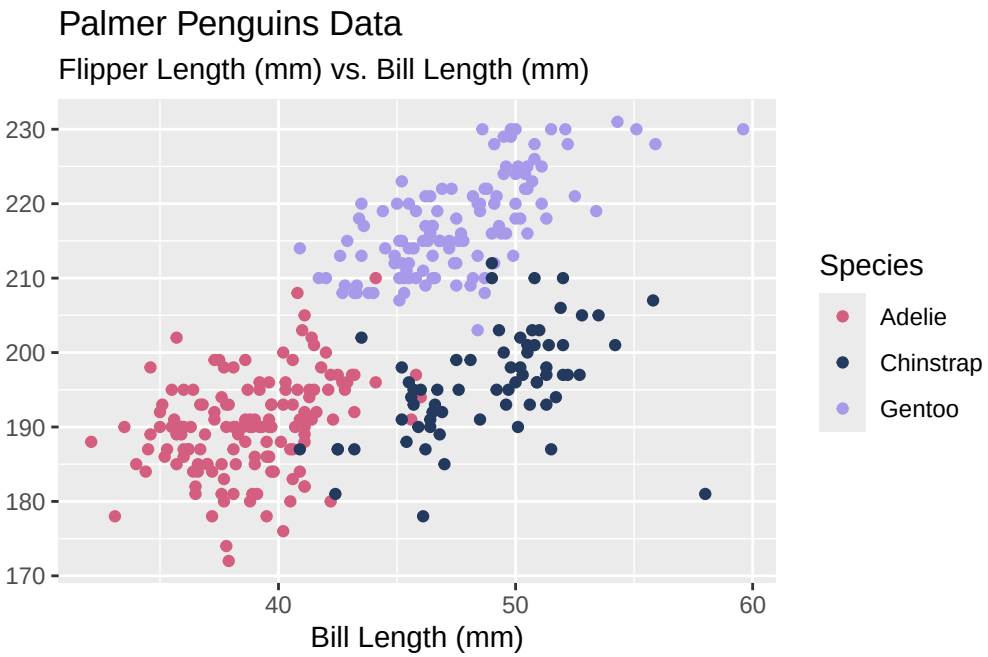

Figure 2.5: Palmers Penguins Data

When applying the Elbow Method, the resulting plot in Figure 2.5 shows a gradual decrease in WCSS rather than a distinct bend. As discussed previously, this makes it difficult to clearly identify a clear optimal number of clusters, since there is no obvious point where the reduction in error slows significantly. As a result, the method provides little guidance and requires subjective interpretation.

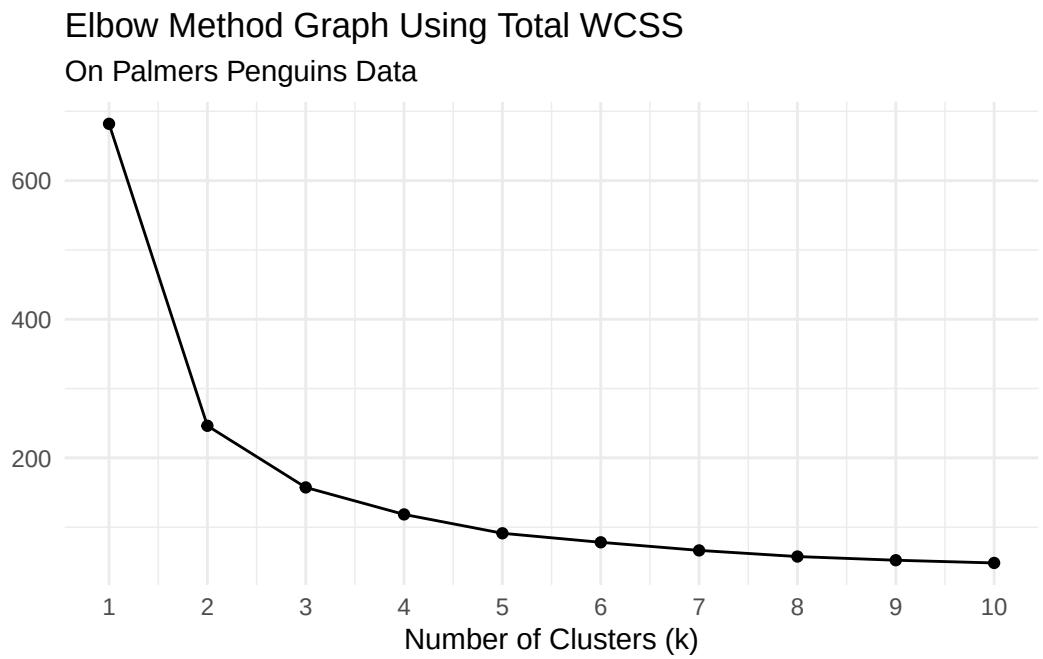

Figure 2.6: Elbow Method on Palmers Penguins Data

The Silhouette Score, shown in Figure 2.6, suggests that two clusters are optimal, as this value produces the highest average silhouette score. According to this metric, the data is best partitioned into two groups. However, this result conflicts with the known structure of the dataset, which contains three distinct species. This discrepancy highlights the method's tendency to favor solutions that maximize separation.

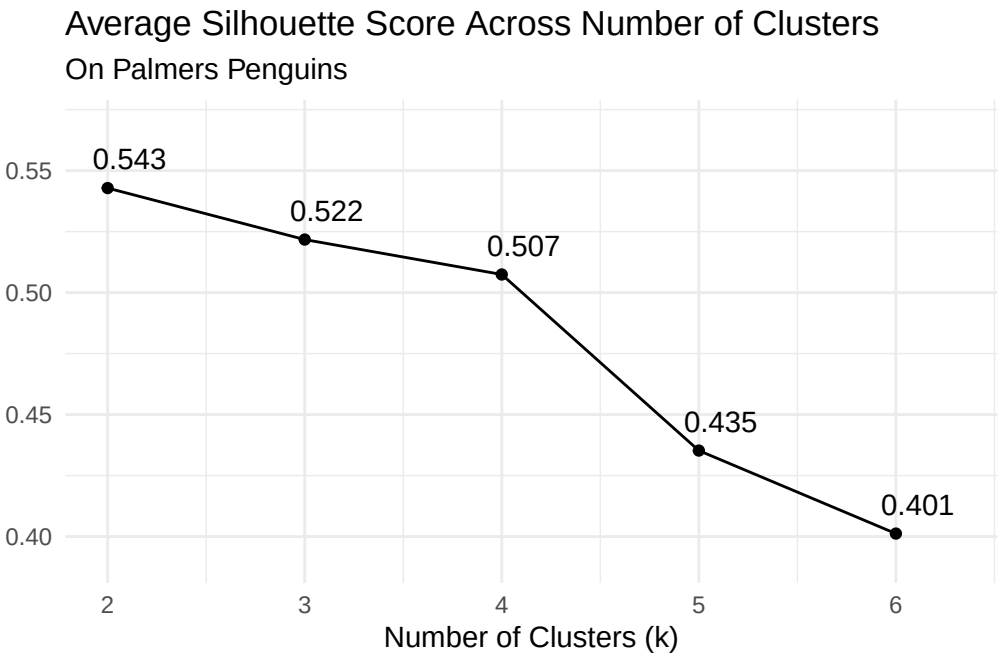

Figure 2.7: Silhouette Score on Palmers Penguins Data

In contrast, the CRAB Score correctly identifies three clusters as the optimal solution with a score of 0.0048. The next lowest score occurs at $k = 2$, which is consistent with the alternative methods. Although the "pinching point" appears less visually pronounced in this example, the local minimum at $k = 3$ provides a clear and objective indication of the suggested number of groupings. More importantly, this result aligns with the known structure of the dataset, as defined by the species label.

Table 2.1: CRAB Performance on Palmers Penguins Table Line Graph

k	Score	Time (seconds)	Best Cluster (k)
2	0.0073200	52.255	
3	0.0048351	48.618	BEST
4	0.1260321	46.879	
5	0.0932065	42.886	
6	0.0937262	36.619	

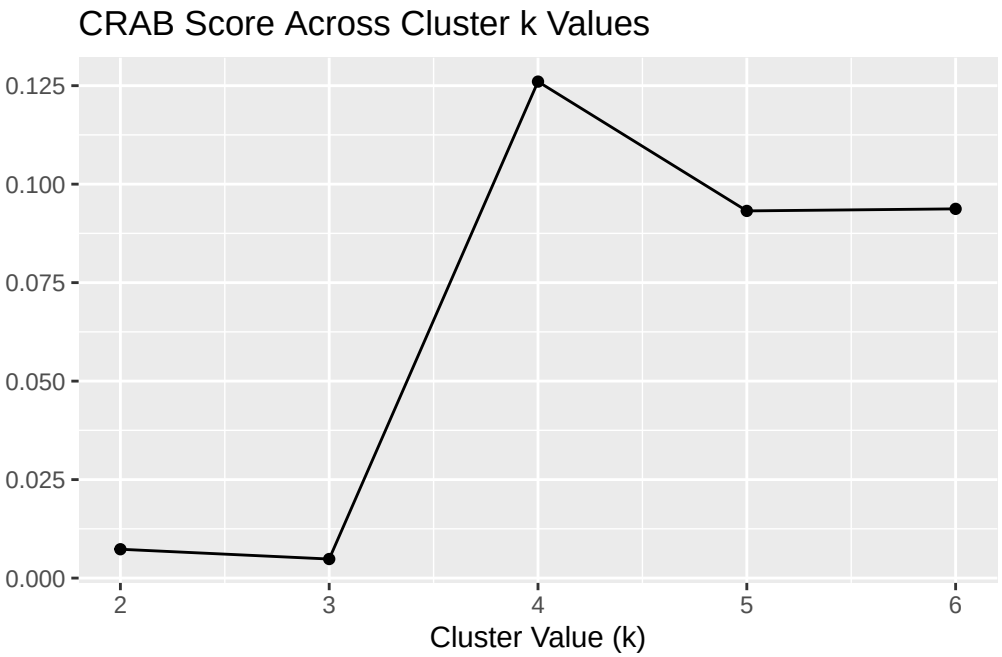

Figure 2.8: CRAB Performance on Palmers Penguins Data

Chapter 4 goes more into depth through a series of simulations that demonstrate CRAB's performance across various datasets, highlighting its ability to identify the true structure of the data. The chapter uses boxplots and other visualizations to illustrate how CRAB performs.

Chapter 3

THE CRAB PACKAGE IN R

3.1 Package Goal

After introducing the CRAB algorithm and outlining its step-by-step logic, we now turn to its practical implementation in R. The cRab package provides a reliable, automated framework for calculating the CRAB score. It emphasizes a tidy and efficient workflow that aligns seamlessly with the tools and conventions R users already rely on. By formalizing this process into an accessible package, cRab offers a rigorous yet intuitive toolkit for validating cluster structures without the need for complex, or custom syntax.

3.2 Design Philosophy and Ecosystem Integration

The cRab package extends the tidyclust ecosystem (cite) and builds on the core principles of the tidymodels (cite) framework. It treats clustering as an integrated step of a machine learning workflow rather than a standalone procedure. This architectural choice keeps the package familiar to modern data scientists by relying exclusively on standard R objects like tibbles and parsnip-style models. By adopting this "recipe" style, cRab fits directly into existing data pipelines without requiring users to learn new data formats.

The core logic of cRab uses the native R pipe (|>) (cite), which establishes a baseline requirement of R Version 4.1. While this introduces a specific version constraint, it prioritizes the long-term stability and readability of the codebase. By utilizing the native pipe, the package reduces external dependencies, such as the magrittr package (cite) which houses the magrittr operator (%>%). As a result, the package maintains a lighter installation footprint and produces more transparent error messages during debugging. Together, these design

choices promote consistent and easy to follow syntax as the R language continues to evolve.

3.3 Functional Pipeline

The cRab package architecture leverages a workflow that preserves the original data and builds a working object that updates as the analysis progresses. Unlike traditional R functions that return immediate transformations of data, cRab employs a system that structures the analysis into three logical phases: Initialization, Configuration, and Execution (Figure 3.1).

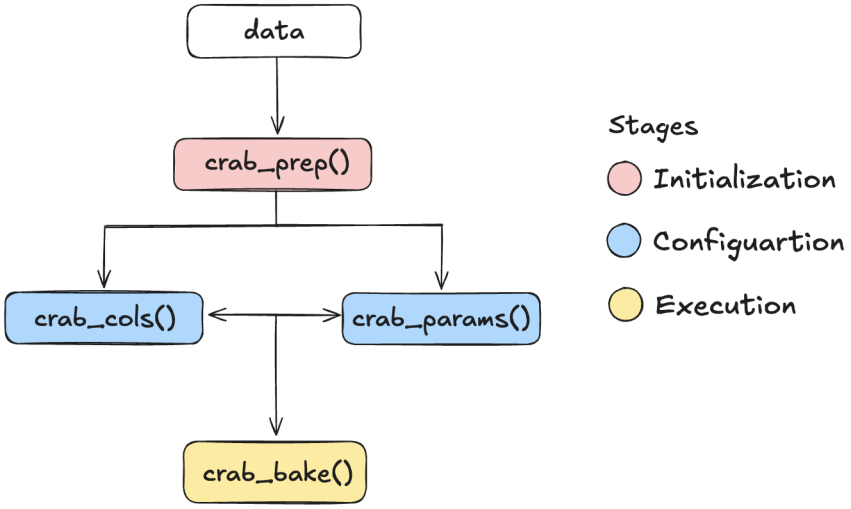

Figure 3.1: cRab Package Workflow Diagram

3.3.1 Phase I: Initialization

The pipeline begins with the crab_prep object, which serves as the central container for the entire validation process. Upon initialization, the package generates a metadata-rich list that stores the raw data alongside a set of empty slots reserved for variable selection and hyperparameters. By storing information within this object, cRab facilitates a precomputation phase where it evaluates and records the structure of the data. This setup keeps later stages of the pipeline computationally efficient and consistent by avoiding repeated computation of the same information.

```r
iris |>
  crab_prep()
```

```
$data

$original_cols
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"

$specified_cols
NULL

$specified_params
list()
```

Figure 3.2: crab_prep() Example Output

3.3.2 Phase II: Configuration

After initialization, the workflow enters the Configuration Stage. To improve flexibility and readability, cRab splits configuration into two modular functions: crab_cols() and crab_params(). The first function, crab_cols(), manages variable selection, allowing the user to define which features contribute to distance metrics and clustering stability. It populates these choices in the specified_cols element within the crab_prep object (Figure 3.3).

```r
iris |>
  crab_prep() |>
  crab_cols(c("Petal.Length", "Petal.Width"))
```

```
$data

$original_cols
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"

$specified_cols
[1] "Petal.Length" "Petal.Width"

$specified_params
list()
```

Figure 3.3: crab_cols() Example Output

Complementing this, crab_params() defines the hyperparameters required for the Buddies and Rivals logic, including subsampling rates and number of iterations. This step updates the specified_params slot (Figure 3.4).

```
iris |>
  crab_prep() |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans")
```

```
$data

$original_cols
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"

$specified_cols
NULL

$specified_params
$specified_params$min_k
[1] 3

$specified_params$max_k
[1] 5

$specified_params$num_subsamples
[1] 10

$specified_params$subsample_prop
[1] 0.8

$specified_params$cluster_method
[1] "kmeans"
```

Figure 3.4: crab_params() Example Output

These two functions operate independently and in any order. As a result, a researcher can call crab_cols() before or after crab_params(), and they can overwrite previous settings without affecting the underlying data. Each call updates only its designated slot within the crab_prep object, which keeps the workflow modular and transparent.

```r
# option 1
iris |>
  crab_prep() |>
  crab_cols(c("Petal.Length", "Petal.Width")) |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans")


# option 2
iris |>
  crab_prep() |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans") |>
  crab_cols(c("Petal.Length", "Petal.Width"))
```

```
$data

$original_cols
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"

$specified_cols
[1] "Petal.Length" "Petal.Width"

$specified_params
$specified_params$min_k
[1] 3

$specified_params$max_k
[1] 5

$specified_params$num_subsamples
[1] 10

$specified_params$subsample_prop
[1] 0.8

$specified_params$cluster_method
[1] "kmeans"
```

Figure 3.5: Configuration Stage Examples

Before moving to the computationally intensive Execution phase, cRab implements a rigorous

validation layer. Each function checks that the input object contains the required metadata and valid data types. The package also provides educational warnings to support correct usage. For instance, if a user sets $k = 1$, a scenario that is mathematically trivial for clustering stability, the package does not stop the execution with an error. Instead, it issues a warning that $k = 1$ does not produce meaningful cluster results. This approach encourages mroe informed choices while maintaining workflow continuity.

3.3.3 Phase III: Execution

The workflow transitions from specifications to results exclusively during the Execution Stage via crab_bake(). At this phase, the function reconciles the stored metadata and applies the configured logic to the raw data. It undoubtedly handles the heavy lifting by performing iterative subsampling and calculation of the final CRAB scores. These computations return a tidy data frame that summarizes cluster stability in a clear and reproducible format.

```
iris |>
  crab_prep() |>
  crab_cols(c("Petal.Length", "Petal.Width")) |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans") |>
  crab_bake()
```

Table 3.1: Execution Stage Example

k	Score	Time (seconds)	Best Cluster (k)
2	0.0001581	21.969	BEST
3	0.0002594	44.644	
4	0.0941276	21.463	
5	0.1022005	20.908	

3.4 Computational Strategy and Resource Management

The cRab pipeline relies on Buddies Matrices with an $n \times n$ structure, which causes the computational cost of cluster validation to grow quadratically as the dataset size increases. To maintain performance across different machines, the package maximizes throughput through parallelism while limiting memory usage in its core data structures.

To accelerate the iterative Buddies and Rivals computations, cRab utilizes the doParallel and foreach libraries. Parallelization significantly reduces the total execution time by distributing iterations across multiple CPU cores, but consequently also increases memory usage since each core requires a copy of the global environment.

To prevent system overload, the package applies a conservative core reservation strategy by default:

$$\text{Cores}_{\text{Active}} = \max(1, \text{detectCores}() - 5)$$

By reserving several cores for background processes, the system remains responsive during execution.

The primary computational bottleneck arises during the construction of the Buddies Matrix, which has a time and space complexity of $O(n^2)$, where n represents the number of observations in each subsample. As n increases, storing all pairwise relationships can exceed available RAM and lead to memory errors.

To address this, cRab uses a more efficient storage strategy. The package reduces redundancy by storing only unique relationships. By utilizing the symmetry of the matrix and keeping only the upper triangle of indices, it effectively cuts storage requirements roughly in half. Additionally, the package applies sparse indexing, tracking only relationship between observations present in each subsample.

These optimizations reduce memory demands and allow larger datasets to run on standard hardware without sacrificing the accuracy of the CRAB score.

3.5 Synthetic Data Generation

While cRab is designed to validate clusters in real-world datasets, controlled simulations provide a way to evaluate its sensitivity when the true cluster structure is known. To support this, the package includes simulate_mvn(), a utility that allows users to benchmark the CRAB score on datasets with predefined cluster properties before applying the tool to ambiguous or unlabeled data.

The simulation framework uses Multivariate Normal (MVN) distributions because they offer precise control over cluster separation, orientation, and density. By manipulating the covariance structure, users can introduce varying levels of overlap and geometric distortion, creating a range of clustering scenarios.

The simulate_mvn() function generates data using three primary user inputs: the total number of observations (n), the desired number of clusters (k), and a variance scalar. These parameters allow users to control how distinct or overlapping the clusters appear.

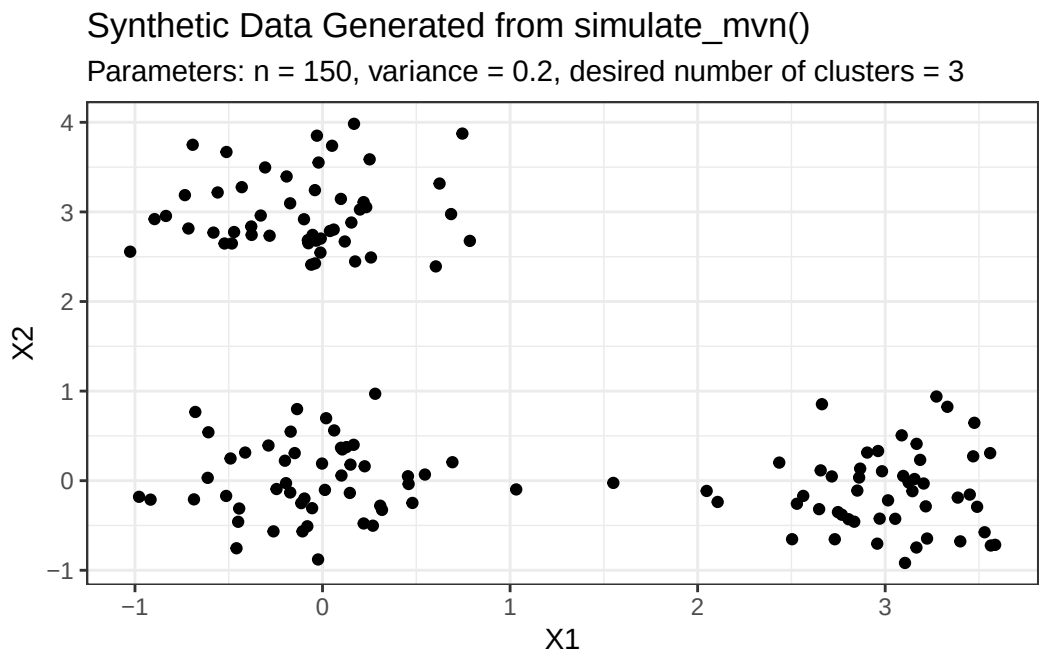

Figure 3.6: Synthetic Data Generated from simulate_mvn()

For a single simulated dataset, the workflow follows the standard pipeline (Figure 3.7). In this scenario, simulate_mvn() generates the data, which then passes through standard preprocessing and parameter specification layers before applying the CRAB algorithm.

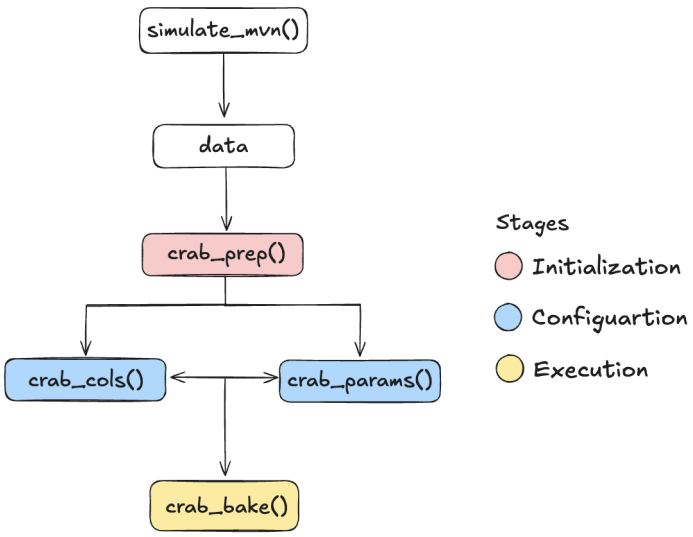

Figure 3.7: Singular Synthetic Dataset Generation with CRAB Algorithm Workflow

Below is an example code chunk:

```
# data from simulate_mvn()
data |>
  crab_prep() |>
  crab_cols(c("X1", "X2")) |>
  crab_params(min_k = 2, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans") |>
  crab_bake()
```

When evaluating performance across multiple runs or varying noise levels, the workflow shifts to a simulation focused approach (Figure 3.8). One option mirrors the standard pipeline by initializing with crab_prep(NULL) and specifying generative parameters through crab_sim_params(), such as the number of runs, variance, and sample size. The repeat_simulations() function then replaces crab_bake() and applies the CRAB algorithm across multiple simulated datasets with consistent settings. Alternatively, users can bypass the pipeline entirely and pass all parameters into repeat_simulations() directly.

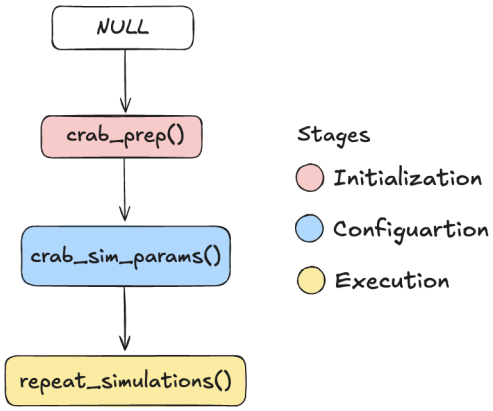

Figure 3.8: Multiple Synthetic Dataset Generation with CRAB Algorithm Workflow

To illustrate these options, consider the following implementations:

```r
# option 1 -- pipeline format
crab_prep(data = NULL) |>
  crab_sim_params(min_k = 2, max_k = 5, num_subsamples = 10,
                  num_runs = 2, variance = c(0.3, 0.4),
                  cluster_method = "kmeans") |>
  repeat_simulations()

# option 2 -- directly passing in parameters
repeat_simulations(min_k = 2, max_k = 5, num_subsamples = 10,
                   num_runs = 2, variance = c(0.3, 0.4),
                   cluster_method = "kmeans")
```

Chapter 4 explores the results of these simulations in greater detail.

3.5.1 Point Diagnostics

In addition to providing a global measure of cluster stability, the cRab framework includes diagnostic tools to identify which observations drive that stability or, more importantly, its instability. The point contribution() function allows for a granular, observation-level analysis by quantifying and visualizing how each individual data point impacts the final CRAB score.

In practice, raw instability values are difficult to interpret. Although they are mathematically meaningful, their scale often varies across different datasets and dimensionalities, making visualization hard to read. To improve interpretability, point_contribution() applies a sigmoid transformation that maps contributions onto a bounded [0, 1] scale. This transformation standardizes the values and makes relative differences easier to interpret.

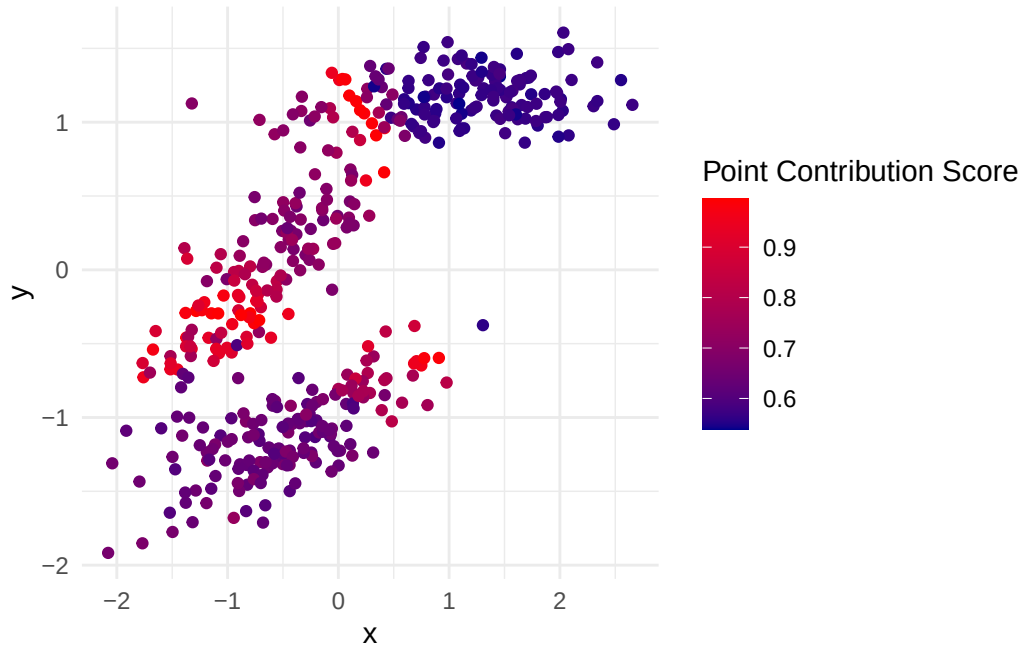

Figure 3.9: Point Contribution Example

The transformation highlights observations that act as "swing voters" in the clustering structure. These points are typically located near cluster boundaries, making them the most likely to switch cluster assignments across subsamples. Figure 3.9 illustrates this behavior where high contribution points appear as bright red regions concentrated near decision boundaries. In contrast, observations that don't change between clusters are labeled in a dark blue color and have a contribution score closer to 0. By isolating these observations, the function helps identify which data points disproportionately influence the CRAB score.

Chapter 4

SIMULATION STUDY

test w non spherical data too

4.1 Non-Spherical Data

Chapter 5

CONCLUSION

- restate and brag
- #better than everybody else
- future work
- note that this was only used for k-means ## Computational Details

The results in this paper were obtained using R 4.4.1. R itself and all packages used are available from the Comprehensive R Archive Network (CRAN) at https://CRAN.R-project.org/.

REFERENCES

APPENDICES

Appendix A

SIMULATE_MVN()

```
# enter functions here
```